

```
def assoc[A,B,C] (p: (A, (B,C))): ((A,B), C) =
  p match { case (a, (b, c)) => ((a,b), c) }
```

Using these combinators, product and assoc, the law of associativity for applicative functors is as follows:

```
product(product(fa,fb),fc) == map(product(fa, product(fb,fc)))(assoc)
```

Note that the calls to product are associated to the left on one side and to the right on the other side of the == sign. On the right side we're then mapping the assoc function to make the resulting tuples line up.

12.5.3 *Naturality of product*

Our final law for applicative functors is *naturality*. To illustrate, let's look at a simple example using Option.

Listing 12.6 Retrieving employee names and annual pay

```
val F: Applicative[Option] = ...

case class Employee(name: String, id: Int)
case class Pay(rate: Double, hoursPerYear: Double)

def format(e: Option[Employee], pay: Option[Pay]): Option[String] =
  F.map2(e, pay) { (e, pay) =>
    s"${e.name} makes ${pay.rate * pay.hoursPerYear}"
  }

val e: Option[Employee] = ...
val pay: Option[Pay] = ...
format(e, pay)
```

Here we're applying a transformation to the *result* of map2—from Employee we extract the name, and from Pay we extract the yearly wage. But we could just as easily apply these transformations separately, before calling format, giving format an Option[String] and Option[Double] rather than an Option[Employee] and Option[Pay]. This might be a reasonable refactoring, so that format doesn't need to know the details of how the Employee and Pay data types are represented.

Listing 12.7 Refactoring format

```
val F: Applicative[Option] = ...

def format(name: Option[String], pay: Option[Double]): Option[String] =
  F.map2(e, pay) { (e, pay) => s"$e makes $pay" }

val e: Option[Employee] = ...
val pay: Option[Pay] = ...

format(
  F.map(e) (_.name),
  F.map(pay)(pay => pay.rate * pay.hoursPerYear))
```

format now takes the employee name as a Option[String], rather than extracting the name from an Option[Employee]. Similarly for pay.

We're applying the transformation to extract the name and pay fields before calling `map2`. We expect this program to have the same meaning as before, and this sort of pattern comes up frequently. When working with `Applicative` effects, we generally have the option of applying transformations *before* or *after* combining values with `map2`. The naturality law states that it doesn't matter; we get the same result either way. Stated more formally,

```
map2(a,b) (productF(f,g)) == product(map(a) (f), map(b) (g))
```

Where `productF` combines two functions into one function that takes both their arguments and returns the pair of their results:

```
def productF[I,O,I2,O2](f: I => O, g: I2 => O2): (I,I2) => (O,O2) =
  (i,i2) => (f(i), g(i2))
```

The applicative laws are not surprising or profound. Just like the monad laws, these are simple sanity checks that the applicative functor works in the way that we'd expect. They ensure that `unit`, `map`, and `map2` behave in a consistent and reasonable manner.



EXERCISE 12.7

Hard: Prove that all monads are applicative functors by showing that if the monad laws hold, the `Monad` implementations of `map2` and `map` satisfy the applicative laws.



EXERCISE 12.8

Just like we can take the product of two monoids `A` and `B` to give the monoid `(A, B)`, we can take the product of two applicative functors. Implement this function:

```
def product[G[_]](G: Applicative[G]):
  Applicative[({type f[x] = (F[x], G[x])})#f]
```



EXERCISE 12.9

Hard: Applicative functors also compose another way! If `F[_]` and `G[_]` are applicative functors, then so is `F[G[_]]`. Implement this function:

```
def compose[G[_]](G: Applicative[G]):
  Applicative[({type f[x] = F[G[x])})#f]
```



EXERCISE 12.10

Hard: Prove that this composite applicative functor meets the applicative laws. This is an extremely challenging exercise.